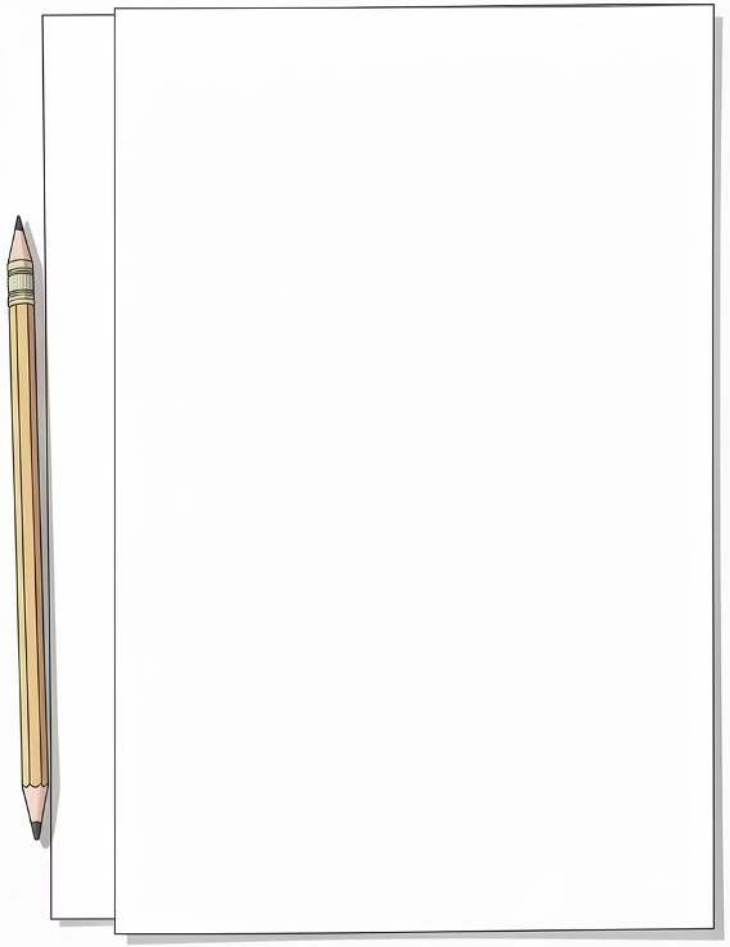
KISS — Keep It Simple, Stupid

Principios de Desarrollo de Software

Juan José Acevedo · Daniel Márquez
Universidad EAFIT — Ingeniería de Software 2026



Made with GAMMA

¿Qué es KISS?

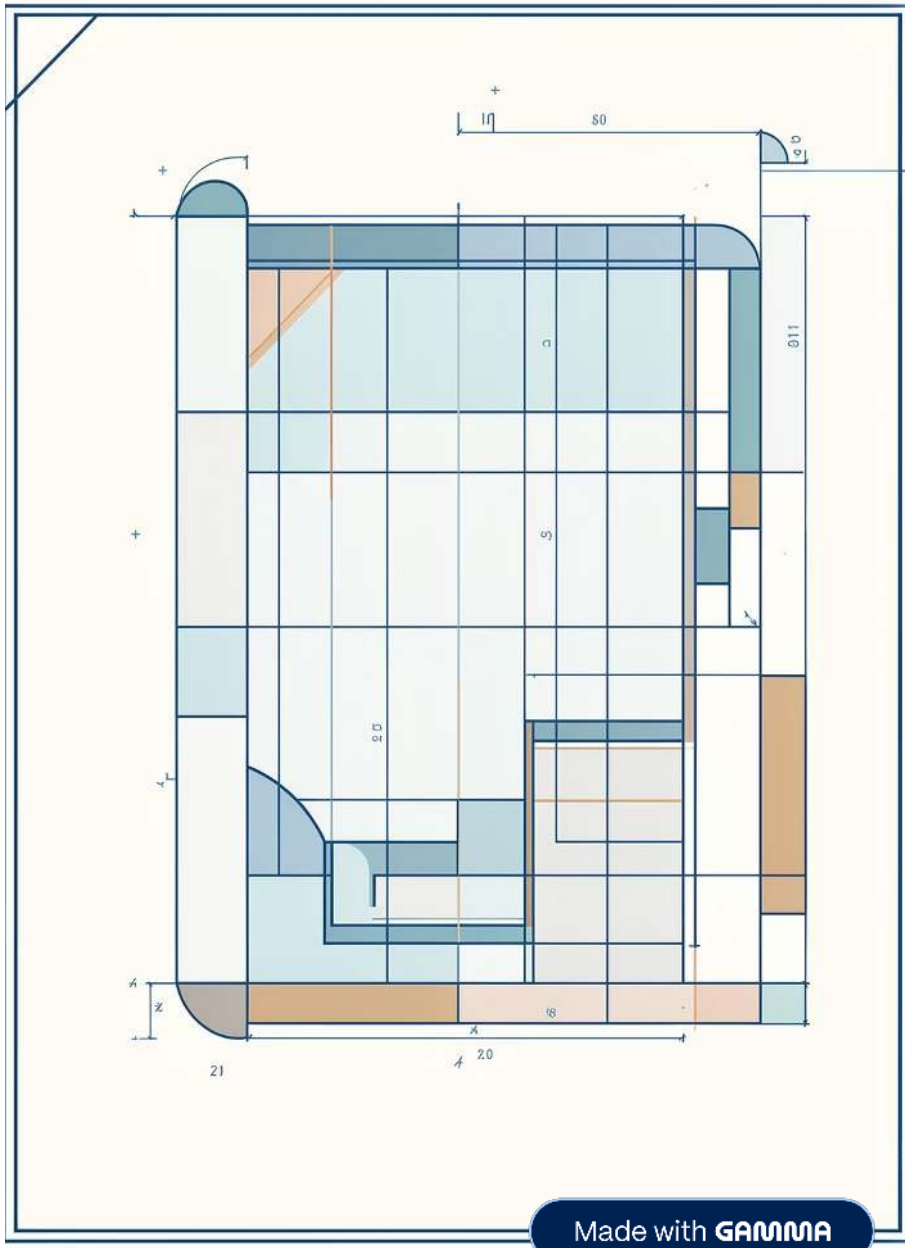
> "La mayoría de los sistemas funcionan mejor si se mantienen simples en lugar de hacerlos complejos. La simplicidad debe ser un objetivo clave en el diseño, y la complejidad innecesaria debe evitarse."
>
> — Kelly Johnson, ingeniero de Lockheed, años 60

La idea central

Si algo puede hacerse simple, **hazlo simple**. Si hay dos soluciones, elige la más fácil de entender. No se trata de hacer las cosas triviales, sino de no añadir complejidad sin razón.

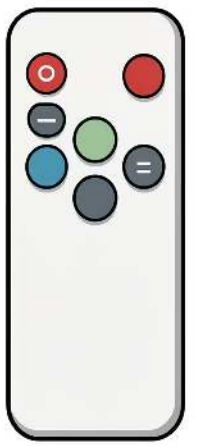
El origen del principio

Kelly Johnson fue el ingeniero jefe del programa de aviones espía U-2 y SR-71 en Lockheed. Su filosofía: los aviones debían poder ser reparados en campo con herramientas básicas. La misma lógica aplica al software.



Made with GAMMA

La Analogía del Control Remoto



Un control remoto con 80 botones hace exactamente lo mismo que uno con 5. El de 80 confunde; el de 5 cualquiera lo usa sin instrucciones.

Lo mismo pasa en el código: **más líneas no significa mejor código.** La complejidad adicional no agrega valor — solo agrega fricción.

Made with GAMMA

¿Por Qué Importa KISS?

Se lee más de lo que se escribe

El código se escribe una vez pero se lee decenas de veces. La claridad es una inversión a largo plazo.

Menos bugs

El código simple tiene menos lugares donde los errores pueden esconderse. Menos complejidad, menos superficie de fallo.

Más fácil de mantener

Modificar código simple es directo. Modificar código complejo requiere entender toda la cadena de dependencias.

Onboarding sin dolor

El equipo nuevo lo entiende sin necesitar explicaciones extensas ni documentación adicional.

Ahorra tiempo y dinero

La complejidad innecesaria tiene un costo real: horas de depuración, reuniones de aclaración y retrasos en entregas.

Made with GAMMA

❌ Sin Aplicar KISS

EJEMPLO EN PYTHON

Verificar si un número es par — versión compleja

```python
def es_par(numero):
    if numero % 2 == 0:
        resultado = True
    else:
        resultado = False
    return resultado
```

¿Qué está mal aquí?

Esta función hace algo muy simple, pero lo expresa de forma innecesariamente larga:

- Crea una variable `resultado` que no aporta nada
- Usa un bloque `if/else` para asignar un booleano que ya existe
- Tiene 5 líneas donde bastaría 1
- Un lector nuevo tarda más en entenderlo

⚠ Más líneas de código no equivalen a mejor código. Equivalen a más carga cognitiva.

Made with GAMMA

✅ Aplicando KISS

EJEMPLO EN PYTHON

Verificar si un número es par — versión KISS

```
def es_par(numero):
    return numero % 2 == 0
```

¿Por qué es mejor?

→ Mismo resultado

La función hace exactamente lo mismo que la versión anterior.

→ Más claro

Se lee como una oración: "retorna si número módulo 2 es cero".

→ Más directo

Sin variables intermedias innecesarias. Sin ruido visual.

La expresión `numero % 2 == 0` *ya es* un booleano. Devolverlo directamente es la forma más honesta y simple de escribirlo.

Made with GAMMA

KISS en Nuestro Proyecto — Profile Manager

MATCHING.PY

compute_match — Cálculo de compatibilidad

La función `compute_match` calcula el score de compatibilidad entre un candidato y una vacante de forma directa:

- Parsea las habilidades del candidato y la vacante
- Calcula el porcentaje de coincidencia
- Aplica un bonus por afinidad

No usa estructuras complejas ni abstracciones innecesarias — **hace exactamente lo que necesita hacer y nada más.**

API/CLIENT.JS

Cliente HTTP centralizado

`api/client.js` centraliza todas las llamadas HTTP en funciones simples con nombres claros:

- `loginUser()`
- `getHome()`
- `extractProfile()`

En lugar de repetir la lógica de `fetch` en cada página, hay un único cliente simple que todos los componentes usan de la misma manera.

Ambos ejemplos comparten el mismo principio: **una responsabilidad, un lugar, sin repetición innecesaria.** Eso es KISS en la práctica real.

Made with GAMMA

¿Cuándo Es Difícil Aplicarlo?

El síndrome del "por si acaso"

Queremos "prepararnos para el futuro" y agregamos código que nadie pidió. Clases abstractas, interfaces genéricas, configuraciones para casos que quizás nunca lleguen.

Complejidad como señal de calidad

A veces confundimos complejidad con calidad. Pensamos que un código más elaborado demuestra más habilidad. En realidad, lo contrario es más difícil: hacer algo simple.

Patrones grandes para problemas pequeños

Copiamos arquitecturas de grandes sistemas (microservicios, event sourcing, CQRS) para proyectos que no los necesitan. El patrón correcto depende del problema, no de la moda.

La pregunta clave: ¿Puedo explicar esto en una oración? Si la respuesta es no, es una señal de que hay que simplificar.

Made with GAMMA

Resumen

KISS no significa código descuidado

Significa código que **resuelve el problema sin sobrecarga innecesaria**. Un código simple puede ser profundo, robusto y elegante al mismo tiempo.

> "La perfección se alcanza no cuando no hay nada más que agregar, sino cuando no hay nada más que quitar."
>
> — Antoine de Saint-Exupéry

Simple ≠ Trivial

Hacer algo simple requiere más esfuerzo y criterio que hacerlo complejo.

Simple ≠ Limitado

Simple es lo opuesto de *complicado*, no lo opuesto de *potente*.

Made with GAMMA

¿Preguntas?

Gracias por su atención.

Juan José Acevedo

Ingeniería de Software
Universidad EAFIT — 2026

Daniel Márquez

Ingeniería de Software
Universidad EAFIT — 2026

KISS — Keep It Simple, Stupid · Principios de Desarrollo de Software

Made with GAMMA